

DATA301 Project Proposal - Sam Ladbrook

LSH Accelerated Collaborative Filtering for Movie Recommendations

1. Summary

This project proposes using the Amazon Movies and TV Reviews dataset, from the McAuley Lab at UCSD, to investigate the effectiveness of Locality Sensitive Hashing (LSH) as a scalability technique for collaborative filtering (CF). The dataset contains millions of user ratings for movies and TV shows sold on Amazon, providing an insightfully large set of data for a meaningful recommendation system analysis. The research question is: "What is the difference in movie recommendation accuracy and computation runtime between LSH accelerated collaborative filtering and brute-force collaborative filtering when applied to the Amazon Movies and TV reviews dataset?" The project will combine two algorithms from the Mining of Massive Datasets (MMDS) textbook, MinHash and Locality Sensitive Hashing, which will be used to efficiently identify candidate similar items and item based collaborative filtering to then use those candidates to generate ranked movie recommendations. The expected result is a quantitative comparison of recommendation accuracy and runtime between the LSH accelerated pipeline and a brute-force cosine similarity baseline, compared using Precision@k and Root Mean Squared Error (RMSE) on a test set. This result is significant because it demonstrates a practical and scalable approach to recommendation that does not sacrifice meaningful accuracy; directly reflecting how real-world platforms such as Neon handle recommendations at a large scale.

2. Motivation

Subscription streaming platforms such as Neon serve a national audience within New Zealand without the engineering resources of larger services like Netflix and Amazon, yet are faced with the same commercial pressure to deliver accurate recommendations; poor suggestions directly increase likelihood of subscribers canceling their subscription. This would make the scalability of recommendation algorithms a genuinely important practical problem. Brute-force collaborative filtering produces high-quality results but is computationally expensive at scale, which creates a real tradeoff for platforms that operate on tighter infrastructure budgets. In addition to this the algorithm behind this project (MinHash, LSH and collaborative filtering) were among the most interesting topics covered both in lectures and in the MMDS textbook for me personally, especially the idea that approximate nearest-neighbour search through hashing can hugely

decrease computation without having a negative effect on result quality. The findings of this project will answer a question that matters for any engineer building recommendation infrastructure at scale.

3. Background

The Amazon Movies and TV Reviews dataset is part of the Amazon Reviews 2018 collection maintained by the McAuley Lab at UCSD. It contains user reviews spanning May 1996 to September 2018, with each records including a user ID, item ASIN, star rating (1-5), review text and a UNIX timestamp. The 5-core subset (where each user and item has a minimum of five interactions) is used to ensure sufficient density for similarity computation. The dataset shows the classic long-tail distribution common to modern e-commerce; a small number of popular titles has the majority of ratings while most items are rated by relatively few users. Item-based collaborative filtering, as described in MMDS Chapter 9, addresses the recommendation problem by constructing a matrix of user-item ratings and computing cosine similarity between item vectors to identify movies that attract similar audiences. This has a scalability problem as computing the pairwise similarity across items is $O(n^2)$, which means computation becomes very heavy at the scale of this dataset. To address this MinHash (MMDS Chapter 3) can be used to compress each item's set of users into a compact hash using a random hash function, taking advantage of the mathematical property that the probability two MinHash hashes agree on a value equals the Jaccard similarity of the original sets. LSH banding then groups the items with matching hash bands into candidate buckets, which limits the expensive cosine similarity computation to within bucket pairs only. This hugely reduces the number of comparisons required while preserving accuracy of results.

4. Research Question

The research question for this project is: "What is the difference in movie recommendation accuracy and computation runtime between LSH accelerated collaborative filtering and brute-force collaborative filtering when applied to the Amazon Movies and TV reviews dataset?" This is a comparative quantitative question directly relevant to the database because the scale of the Amazon Movies and TV dataset makes brute-force similarity computation genuinely expensive, proving a real need for LSH approximation. The question will be answered by implementing both approaches, generating top-10 recommendations for a held-out set of test users and comparing accuracy using Precision@10 and RMSE against actual held-out ratings. Implementing the MinHash and LSH pipeline directly answers how much accuracy is lost for the computation gain, while implementing the brute-force CF baseline provides the

benchmark against which that tradeoff is measured. Together the two implementations produce a concrete, reliable and quantitative result that directly addresses the question.

5. Design and Methods

The implementation will follow a multiple-step pipeline executed in Python using Dask.

Stage 1 - Data loading and preprocessing: The Movies and TV 5-core JSON dataset will be loaded into a Dask Bag, the same as the approach demonstrated in the sample project code. Records will be filtering to only contain the user ID, item ASIN and star rating fields and then converted to a Dask DataFrame. The data will be split 80/20 into the training and test sets by randomly holding out ratings per user, ensuring every test user has sufficient training history. **Stage 2 - Building user sets:** For each movie in the training set, the set of users who rated it 4 or 5 stars will be computed using a groupby operation producing a dictionary of item-to-user sets that forms the input for both the MinHash hashes and the brute-force cosine similarity vectors. **Stage 3 - Hash generation:** A bank of n hash functions (where n will change across 50, 100 and 200 to measure the effect on accuracy) will be applied to each item's user set. Each hash function takes the form $f(x) = (ax + b) \bmod p$ with randomly generated parameters a and b per function and the minimum hash value across all users kept. This step will be implemented from scratch in Dask. **Stage 4 - LSH banding and candidate generation:** Each hash will be divided up into b bands or r rows. Items whose hashes agree on at least one band are placed into the same candidate buckets, forming approximate nearest neighbors without all pairs being compared. **Stage 5 - Collaborative filtering and evaluation:** Cosine similarity will be computed within each bucket and across all item pairs for the brute-force baseline. For each test user the top-10 recommended items will be generated by aggregating similarity-weighted ratings from their training history. Recommendations will be compared and evaluated against held-out ratings using Precision@10 and RMSE and runtime for each approach will be recorded.

Timetable: Week 1 (May 2-9) - data loading, preprocessing and train/test split. Week 2 (May 10-16) - MinHash implementation and LSH banding (Progress report due May 16). Week 3 (May 17-23) - Brute-force CF baseline and LSH accelerated CF implementation. Week 4 (May 24-30) - evaluation, results table, written report and code cleanup (Submission due May 30). **Potential difficulties and limitations:** The main technical risk is the MinHash implementation; correctly paramertising hash functions and validating that hash similarity converges to Jaccard similarity will be difficult and require lots of testing. Items in long tail of the dataset with very few rating will not be able to produce many user sets making their similarity estimates noisier; this will be a limitation in the written report. Another potential problem could be the dataset size with memory management, Dask partitioning will be used to keep operations within available

memory. Finally the choice of both bands and row parameters directly controls the probability of correctly bucketing similar items and selecting these values involves tradeoffs between time and computation costs that will be discussed as a part of the analysis.

6. References

Leskovec, J., Rajaraman, A., & Ullman, J. D. (2020). Mining of Massive Datasets (3rd ed.). Cambridge University Press.

Lærd Dissertation. (n.d.). *How to structure quantitative research questions*.

<https://dissertation.laerd.com/how-to-structure-quantitative-research-questions.php>

McAuley Lab. (2023). Amazon Reviews 2023 Dataset. University of California San Diego. <http://snap.stanford.edu/data/amazon/productGraph/>

Dask Development Team. (2024). Dask: Library for dynamic task scheduling.

<https://www.dask.org/>

Linden, G., Smith, B., & York, J. (2003). Amazon.com recommendations: Item-to-item collaborative filtering. IEEE Internet Computing

Adomavicius, G., & Tuzhilin, A. (2005). Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions. IEEE Transactions on Knowledge and Data Engineering

Broder, A. Z. (1997). On the resemblance and containment of documents. Proceedings of the Compression and Complexity of Sequences